

CSCC63 --- Computability & computational complexity

Week 3 Tutorial

More practice with Turing machines, decidable and recognizable languages

Question 1

Enumerators

An enumerator for a language $L \subseteq \Sigma^*$ is a TM E_L that, over time, outputs all the strings of L and nothing else.

We will make the enumerator output the strings in L by having it write them on tape 1, each preceded and followed by the special symbol $\#$ (not in Σ).

It is OK for the enumerator to output the same string more than once.

Question 1 (cont'd)

So, E_L is a multitape TM:

- Initially each tape is blank with its head in the leftmost cell, and E_L is in its initial state.
- The computation of E_L is a sequence of configurations $C_1 \vdash C_2 \vdash \dots \vdash C_i \vdash \dots$ so that:

(a) Tape 1 always contains a string of the form

$$\#x_1\#x_2\#x_3 \dots,$$

where $x_i \in \Sigma^*$ and $\# \notin \Sigma$

(b) if $i < j$ the string on tape 1 in C_i is a prefix of the string on tape 1 in C_j

(c) for any $x \in \Sigma^*$, the string $\#x\#$ is a substring of the string on tape 1 in some C_i if and only if $x \in L$.

Question 1 (cont'd)

Enumerators provide useful alternative characterizations of recognizable and decidable languages.

Theorem 1 (Sipser, Theorem 3.21): L is recognizable if and only if there is an enumerator for L .

Theorem 2 (Sipser, Exercise 3.18): L is decidable if and only if there is an enumerator for L that outputs the elements of L in shortlex order.*

*shorter strings before longer ones; strings of the same length in lexicographic (dictionary) order.

Question 1 (cont'd)

Prove Theorem 1

L is recognizable if and only if there is an enumerator for L .

[if] L -recognizer (given enumerator $\underline{E_L}$ for $\underline{L}$):
on input x , start E_L ; if it ever outputs x , accept.

[only if]

Let $x_0, x_1, x_2, \dots$ be the strings of Σ^* in shortlex order.

L -enumerator (given recognizer $\underline{M}$ for $\underline{L}$):

dovetail through all pairs of natural numbers;

when considering pair (i, j) , run M on string x_i for j steps;

if M accepts, output x_i

Question 1 (cont'd)

Prove Theorem 2:

L is decidable if and only if there is an enumerator for L that outputs the elements of L in shortlex order.

[if] If L is finite, then it is obviously decidable.

If L is infinite:

L -decider (given ordered enumerator $\underline{E_L}$ for $\underline{L}$):

on input x :

start E_L

for each y output by E_L **do**

if $y = x$ **then** accept

if y is after x in shortlex order **then** reject

Question 1 (cont'd)

Prove Theorem 2:

L is decidable if and only if there is an enumerator for L that outputs the elements of L in shortlex order.

[only if]

Ordered L -enumerator (given decider $\underline{M}$ for $\underline{L}$):

for each $x \in \Sigma^*$, in shortlex order:

run M on x

if M accepts **then** output x

Question 2

Prove: Every infinite recognizable language has an infinite decidable subset.

Do not post on the internet without the copyright owner's written permission

Question 2

Prove: Every infinite recognizable language has an infinite decidable subset.

Hint: Use the characterizations of recognizable and decidable languages in Theorems 1 and 2.

Do not post on the internet without the copyright owner's written permission

Question 2

Prove: Every infinite recognizable language has an infinite decidable subset.

Hint: Use the characterizations of recognizable and decidable languages in Theorems 1 and 2.

Let L be an infinite recognizable language.

By Theorem 1, there is an enumerator E_L of L .

Use E_L to construct an ordered enumerator of an infinite subset L' of L .

By Theorem 2, L' is decidable.

Question 2

Ordered enumerator of L' :

$last := \varepsilon$

start the enumerator E_L

for each string x output by E_L **do**

if x is after $last$ in shortlex order **then**

 output x

$last := x$

Why is this an ordered enumerator?

Why does it output a subset of L ?

Why does it output an infinite set?

Question 3

A pair of positive integers are called **twin primes** if they are prime numbers that differ by 2.

E.g., 11 and 13

2549 and 2551

Twin Prime Conjecture: There are infinitely many pairs of twin primes. (Long-standing open question!)

The largest known pair of twin primes is:

$$2,996,863,034,895 \cdot 21,290,000 \pm 1.$$

For all anyone knows, these may be the last two!

Or there could be more, maybe even infinitely many more.

Question 3 (cont'd)

Is the set A decidable?

$$A = \{\langle n \rangle : n \text{ and } n + 2 \text{ are twin primes}\}$$

Yes.

By Church's thesis there is a TM that computes:

$\text{Prime}(x)$

for $i := 2$ **to** $\lfloor \sqrt{x} \rfloor$ **do**

if i divides x **then return false**

return true

So we can decide A by testing that both $\text{Prime}(x)$ and $\text{Prime}(x + 2)$ return **true**.

Question 3 (cont'd)

Is the set B decidable?

$B = \{\langle m \rangle : \text{for all } n > m, n \text{ and } n + 2 \text{ are not twin primes}\}$

Yes!

- If the Twin Prime Conjecture is false, then there is a largest pair of twin primes, say p and $p + 2$, and B is the set of (encodings of) all integers $\geq p$, which is decidable.
- If the Twin Prime Conjecture is true, then B is the empty set, which is also decidable.

Either way, B is decidable: there is a TM that decides it, even though we can't pin down which of two possibilities!

Question 4

Prove: The set of decidable languages is closed under concatenation.

That is: If L and L' are decidable, then so is

$$LL' = \{y: \exists x \in L, \exists x' \in L' \text{ s. t. } xx' = y\}$$

Hint: Take advantage of non-determinism!

Do not post on the internet without the copyright owner's written permission

Question 4 (cont'd)

Guess the place where y is split into the two substrings x and x' whose concatenation makes y .

Then use deciders for L and L' to verify that $x \in L$ and $x' \in L'$.

How to guess where to split y :

Keep taking steps moving R , at each point making a nondeterministic choice:

- this is the end of x , or
- this is not yet the end of x

Question 4 (cont'd)

In more detail: Use a 3-tape nondeterministic TM; initially the input is on tape 1, the other two are empty.

1. [Choose x] Make a nondeterministic choice:
 - mark the present position of the tape 1 head, and copy to tape 2 the string on tape 1 from its leftmost cell up to but not including the marked position; or
 - move the tape 1 head one position to the right and go back to step 1.
2. [Choose x'] Copy on tape 3 the string on tape 1 from the marked position to the end.
3. [Verify] Run a decider for L on the string on tape 2. If it accepts then run a decider for L' on the string on tape 2. If both accept, then accept; else reject.

Question 4 (cont'd)

Is the set of recognizable languages closed under concatenation?

Is it OK to use recognizers instead of deciders in the preceding argument?

Yes!

Do not post on the internet without the copyright owner's written permission